UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Branch: MCA (Data Science)	**Semester:** 2nd
Student Name: Drishti Bhatia	**UID:** 25MCD10008
Subject Name: Technical Training - I Lab	**Subject Code**: 25CAP-652
Section/Group: 25MCD- KAR1	**Date of Performance: 27**-02-2026

Experiment No.: 5

1. **Aim of the practical:**
   To gain hands-on experience in creating and using cursors for row-by-row processing in a database, enabling sequential access and manipulation of query results for complex business logic. **(Company Tags: Infosys, Wipro, TCS, Capgemini)**

2. **Tools Used:** PostgreSQL

3. **Objectives of the practical:**
   - **Sequential Data Access:** To understand how to fetch rows one by one from a result set using cursor mechanisms.
   - **Row-Level Manipulation:** To perform specific operations or calculations on individual records that require conditional procedural logic.
   - **Resource Management:** To learn the lifecycle of a cursor: Declaring, Opening, Fetching, and importantly, Closing and Deallocating to manage system memory.
   - **Exception Handling:** To handle cursor-related errors and performance considerations during large-scale data iteration.

4. **Theory:**
   While SQL is generally set-oriented, certain tasks require a procedural approach where we process one row at a time. This is where **Cursors** are used:
   1. **Cursor Types:** Cursors can be Implicit (managed by the system) or Explicit (defined by the developer). They can also be Forward-Only (moving only toward the end) or Scrollable (moving back and forth).
   2. **The Lifecycle: * DECLARE:** Defines the SQL query for the cursor.
      - **OPEN:** Executes the query and establishes the result set.
      - **FETCH:** Retrieves a specific row into variables for processing.
      - **CLOSE:** Releases the current result set.
      - **DEALLOCATE:** Removes the cursor definition from memory.
   3. **Use Case:** Cursors are ideal for generating row-specific reports, updating balances based on complex historical data, or migrating data where each record needs individual validation.

5. Experiment / Practical Steps:

Step 1: Implementing a Simple Forward-Only Cursor

Creating a cursor to loop through an Employee table and print individual records.

Query:

```
--create table
CREATE TABLE employeee (
   emp_id SERIAL PRIMARY KEY,
   emp_name VARCHAR(100),
   salary NUMERIC(10,2),
   experience INT,
   performance_rating NUMERIC(3,1)
);


--insert sample data
INSERT INTO employeee (emp_name, salary, experience, performance_rating)
VALUES
('Amit Sharma', 30000, 2, 3.5),
('Neha Verma', 45000, 5, 4.2),
('Rahul Singh', 60000, 8, 4.8),
('Priya Mehta', 35000, 3, 2.9),
('Karan Malhotra', 70000, 10, 4.5);

 --step1
DO $$
DECLARE
   emp_record RECORD;
   emp_cursor CURSOR FOR
      SELECT emp_id, emp_name, salary FROM employeee;
BEGIN
   OPEN emp_cursor;

   LOOP
      FETCH emp_cursor INTO emp_record;
      EXIT WHEN NOT FOUND;

      RAISE NOTICE 'ID: %, Name: %, Salary: %',
         emp_record.emp_id,
         emp_record.emp_name,
         emp_record.salary;
   END LOOP;

   CLOSE emp_cursor;
END $$;
```

Output:

```
NOTICE:  ID: 1, Name: Amit Sharma, Salary: 31500.00
NOTICE:  ID: 2, Name: Neha Verma, Salary: 54000.00
NOTICE:  ID: 3, Name: Rahul Singh, Salary: 72000.00
NOTICE:  ID: 4, Name: Priya Mehta, Salary: 36750.00
NOTICE:  ID: 5, Name: Karan Malhotra, Salary: 84000.00
DO

Query returned successfully in 392 msec.
```

Step 2: Complex Row-by-Row Manipulation
Using a cursor to update salaries based on a dynamic "Experience-to-Performance" ratio logic.

Query:

```
--step2
DO $$
DECLARE
   emp_record RECORD;
   ratio NUMERIC;
   emp_cursor CURSOR FOR
      SELECT emp_id, salary, experience, performance_rating
      FROM employeee
      FOR UPDATE;
BEGIN
   OPEN emp_cursor;

   LOOP
      FETCH emp_cursor INTO emp_record;
      EXIT WHEN NOT FOUND;

      ratio := emp_record.experience * emp_record.performance_rating;

      IF ratio > 20 THEN
         UPDATE employeee
         SET salary = salary * 1.20
         WHERE CURRENT OF emp_cursor;

      ELSIF ratio BETWEEN 10 AND 20 THEN
         UPDATE employeee
         SET salary = salary * 1.10
         WHERE CURRENT OF emp_cursor;

      ELSE
         UPDATE employeee
         SET salary = salary * 1.05
         WHERE CURRENT OF emp_cursor;
      END IF;
```

```
    END LOOP;

    CLOSE emp_cursor;
END $$;

--view updated table
select * from employeee;
```

Output:

	emp_id [PK] integer	emp_name character varying (100)	salary numeric (10,2)	experience integer	performance_rating numeric (3,1)
1	1	Amit Sharma	33075.00	2	3.5
2	2	Neha Verma	64800.00	5	4.2
3	3	Rahul Singh	86400.00	8	4.8
4	4	Priya Mehta	38587.50	3	2.9
5	5	Karan Malhotra	100800.00	10	4.5

Step 3: Exception and Status Handling
Ensuring the cursor handles empty result sets or termination signals gracefully.

Query:

```
--step3
DO $$
DECLARE
    emp_record RECORD;
    emp_cursor CURSOR FOR
        SELECT emp_id, emp_name FROM employee;
    row_count INT := 0;
BEGIN
    OPEN emp_cursor;

    LOOP
        FETCH emp_cursor INTO emp_record;
        EXIT WHEN NOT FOUND;

        row_count := row_count + 1;

        RAISE NOTICE 'Processing Employee: % - %',
            emp_record.emp_id,
            emp_record.emp_name;
    END LOOP;

    IF row_count = 0 THEN
```

```
        RAISE NOTICE 'No employees found.';
    ELSE
        RAISE NOTICE 'Total Employees Processed: %', row_count;
    END IF;

    CLOSE emp_cursor;

EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Error: %', SQLERRM;
        IF emp_cursor IS NOT NULL THEN
            CLOSE emp_cursor;
        END IF;
END $$;
```

Output:

```
NOTICE:  Processing Employee: 1 - Amit Sharma
NOTICE:  Processing Employee: 2 - Neha Verma
NOTICE:  Processing Employee: 3 - Rahul Singh
NOTICE:  Processing Employee: 4 - Priya Mehta
NOTICE:  Processing Employee: 5 - Karan Malhotra
NOTICE:  Total Employees Processed: 5
DO
```

6. Learning Outcomes:

- **Cursor Implementation:** Students will be able to design, implement, and manage cursors to solve row-wise processing problems.
- **Lifecycle Mastery:** Students will demonstrate the correct syntax for declaring, opening, fetching, and closing cursors.
- **Error Prevention:** Students will understand how to properly handle row-by-row processing exceptions and prevent memory leaks via deallocation.
- **Analytical Thinking:** Students will be able to apply cursor-based logic to solve real-world scenarios like multi-level payroll adjustments or data migrations.